

A
Project Document
on
SMART TRIP PLANNER

Project Document — Presentation 1

Submitted in partial fulfilment of the requirements of the subject
MC03094171 Research Work (Phase-1) / Minor Project
Master of Computer Applications — Semester III

Submitted by (Team No. **[[TEAM NO]])**

Name	Enrollment No.	Role
Udit Mishra	[[ENROLLMENT NO]]	Backend / Database / API
Hardik Vaghela	[[ENROLLMENT NO]]	Frontend / UI
Vraj [[SURNAME]]	[[ENROLLMENT NO]]	Documentation / UML / System Design

Under the guidance of
[[GUIDE NAME]]

Master of Computer Applications
Sardar Vallabhbhai Patel Institute of Technology, Vasad

Academic Year 2026-27
Presentation date: 21 September 2026

TABLE OF CONTENTS

List of Figures.....	ii
List of Tables.....	iii
1. Introduction.....	1
1.1 Existing System.....	1
1.2 Need for the New System.....	2
1.3 Objective of the New System.....	3
1.4 Problem Definition.....	3
1.5 Scope of the Project.....	4
1.6 Core Components.....	5
1.7 Development Tools and Technologies.....	7
1.8 Hardware and Software Requirements.....	9
1.9 Assumptions and Constraints.....	9
2. Requirement Determination & Analysis.....	11
2.1 Functional Requirements.....	11
2.2 Targeted Users.....	15
3. System Design.....	16
3.1 Use Case Diagram.....	16
3.2 Activity Diagram.....	20
3.3 Interaction Diagram.....	24
3.4 Class Diagram.....	27
3.5 Data Dictionary.....	31

LIST OF FIGURES

Figure 1.1: System Architecture of Smart Trip Planner.....	6
Figure 3.1: Use Case Diagram.....	17
Figure 3.2: Activity Diagram — Plan Trip, Part 1 (Generate Plan Preview).....	21
Figure 3.3: Activity Diagram — Plan Trip, Part 2 (Save Trip).....	22
Figure 3.4: Activity Diagram — Registration and Login.....	23
Figure 3.5: Sequence Diagram — Generate Plan Preview.....	25
Figure 3.6: Sequence Diagram — Save Trip with JWT Authentication.....	26
Figure 3.7: Sequence Diagram — Login and Session Restore.....	27
Figure 3.8: Class Diagram — Domain Model.....	29
Figure 3.9: Class Diagram — Services and Provider Interfaces.....	30

LIST OF TABLES

Table 1.1: Core Components (Modules) of the System.....	7
Table 1.2: Development Tools and Technologies.....	7
Table 1.3: Requirements for Development.....	9
Table 1.4: Requirements for End Users.....	9
Table 2.1: Functional Requirements — Authentication.....	11
Table 2.2: Functional Requirements — Destinations and Places.....	12
Table 2.3: Functional Requirements — Planning.....	12
Table 2.4: Functional Requirements — My Trips (login required).....	13
Table 2.5: Functional Requirements — System Functions.....	14
Table 2.6: Non-Functional Requirements.....	14
Table 2.7: Targeted Users.....	15
Table 3.1: Actors.....	16
Table 3.2: Use Case Description — Plan Trip.....	17
Table 3.3: Use Case Description — Save Trip.....	18
Table 3.4: Use Case Description — Login.....	19
Table 3.5: Use Case Description — Discover Nearby Places.....	19
Table 3.6: Class Responsibilities.....	28
Table 3.7: Data Dictionary — users.....	31
Table 3.8: Data Dictionary — destinations.....	31
Table 3.9: Data Dictionary — dailyCost tier (embedded in destinations).....	32
Table 3.10: Data Dictionary — activities.....	33
Table 3.11: Data Dictionary — trips.....	34
Table 3.12: Data Dictionary — itineraryDay (embedded in trips.itinerary).....	35
Table 3.13: Data Dictionary — itineraryItem (embedded in itineraryDay.items).....	35
Table 3.14: Data Dictionary — budget (embedded in trips.budget).....	36
Table 3.15: Relationships and Indexes.....	36
Table 3.16: Worked Budget Example (Manali, 4 days, 2 travellers, standard tier).....	37

1. INTRODUCTION

Smart Trip Planner is a web application in which a traveller selects an Indian destination, enters the travel dates, the number of travellers, a budget tier, the interests of the group and the preferred pace, and receives two results: a day-wise itinerary assembled from the places of that destination and ordered so that travel between consecutive stops is short, and an estimated budget divided into stay, food, local transport and activity fees. The plan can be generated and inspected without an account; saving it to a personal list called My Trips requires registration and login.

This document covers the first three chapters required for Presentation 1 of the subject MC03094171 Research Work (Phase-1) / Minor Project: the introduction to the problem and the proposed system, the determination and analysis of requirements, and the system design expressed through UML diagrams and a data dictionary.

1.1 Existing System

There is no single existing system that a traveller in India uses to plan the days of a trip. Planning is carried out manually by combining several unrelated sources:

- **Travel blogs and video channels** are used to find out which places are worth visiting. The advice is written for the author's own trip length, season and interests, and has to be re-interpreted by every reader.
- **Map applications** are used to check where each place is and how far it is from the next one. The traveller has to enter the places one by one and compare distances mentally; the map does not know how long a visit takes or how many hours of sightseeing fit into one day.
- **Spreadsheets, notes applications or paper** are used to write down the day-wise plan and to add up the expected cost. The arithmetic (rooms $\times$ nights, per-person food $\times$ days, entry fees $\times$ travellers) is done by hand and must be redone whenever the dates or the group size change.
- **Online travel agencies (OTAs) and booking portals** sell flights, hotels and fixed packages. Their purpose is booking, not planning: they show the price of an individual hotel or a complete package but do not build a personalised day-by-day schedule or a complete trip budget for a self-planned trip.

- **Local travel agents** prepare an itinerary on request, but the plan is fixed, tied to the agent's own vendors, and the cost break-up is rarely transparent.

The limitations of this manual approach are concrete:

- Information is scattered over many sources, so planning a four-day trip typically takes several hours of reading and cross-checking.
- The order of visits is decided by guesswork. Places on opposite sides of a town are often put on the same day, which wastes time and transport money.
- There is no check of feasibility: a hand-made plan does not add visit durations and travel times, so days are either overloaded or half empty.
- Budget estimates are incomplete. Travellers usually remember the hotel tariff but forget local transport, food and entry fees, and the per-person share is not calculated.
- The plan is not reusable. It lives in a chat message or a spreadsheet, cannot be regenerated for a different pace or budget tier, and is not stored together with the traveller's other trips.
- Personal interests (for example adventure versus heritage) are not taken into account unless the traveller filters every source manually.

1.2 Need for the New System

A self-planning traveller needs one place where destination knowledge, distance calculation, time budgeting and cost estimation work together. The new system is needed for the following reasons:

- To replace several hours of manual collection with a plan that is generated in a few seconds from curated destination data.
- To make the itinerary feasible by construction: every day is filled only up to a fixed number of hours that depends on the chosen pace, and travel time between stops is counted.
- To reduce unnecessary travel by ordering the stops of a day using the distance between them.

- To give a complete and explainable budget in which every component (stay, food, transport, activities) is visible together with the total and the per-person amount.
- To let the traveller try alternatives — another budget tier, a different pace, other interests — without repeating the manual work.
- To store the final plan in the traveller's account so that it can be opened later, annotated with notes, marked as completed or cancelled, or deleted.

1.3 Objective of the New System

The objectives of Smart Trip Planner are:

1. To maintain a catalogue of Indian destinations with descriptive information, best season, ideal trip length, per-tier daily costs and a list of activities with real coordinates, visit durations and entry fees.
2. To generate a deterministic day-wise itinerary for a selected destination from the travel dates, interests and pace given by the traveller, using a scoring rule and a nearest-neighbour ordering of stops.
3. To estimate the trip budget for three tiers (economy, standard, luxury) using a transparent formula and to present the break-up, the total and the per-person cost in Indian Rupees.
4. To allow a plan to be previewed without login and to be saved, listed, viewed, updated and deleted by a registered traveller, with each traveller able to access only his or her own trips.
5. To integrate external place and distance services (Google Places API and Google Distance Matrix API) through replaceable provider adapters, while remaining fully functional without them.
6. To expose all functionality through a documented REST API with a uniform response format, input validation and standard security measures.

1.4 Problem Definition

Given a destination D with a set of activities A , where every activity has a geographic location, a type, a rating, a typical visit duration, an entry fee and an optional best time of day; and given

a plan input consisting of a start date, an end date (at most 14 days inclusive), the number of travellers (1 to 12), a budget tier, a set of interests and a pace — the system has to:

1. select and distribute activities over the days of the trip so that the sum of visit time and travel time of a day never exceeds the capacity of the chosen pace, activities matching the traveller's interests are preferred, and consecutive stops are close to each other;
2. assign a start time and an end time to every stop, beginning at 09:00 and including a lunch break;
3. compute the expected cost of stay, food, local transport and activities for the whole group and per person; and
4. keep the generated plan, on request, as a trip that belongs to exactly one registered user.

The problem therefore combines a small scheduling and routing problem (solved with a greedy heuristic, which is adequate for the 8 to 10 activities of a destination) with conventional information-system functions: catalogue browsing, authentication and CRUD operations on saved trips.

1.5 Scope of the Project

1.5.1 In scope

- Public browsing of 12 seeded Indian destinations (Manali, Goa, Jaipur, Udaipur, Munnar, Alleppey, Rishikesh, Varanasi, Darjeeling, Jaisalmer, Kutch and Leh-Ladakh) with keyword search, category filter and pagination.
- Destination details: overview, best season, ideal number of days, daily cost table for the three tiers and the list of activities.
- Discovery of nearby places for a destination through the places provider (local data, or Google Places when an API key is configured).
- Trip planning form, generation of the day-wise itinerary and the budget estimate, and preview of the result without login.
- User registration, login, session restore and logout using JSON Web Tokens.
- My Trips: saving a plan, listing saved trips (optionally by status), viewing a trip, editing title, notes and status, and deleting a trip.

- A REST API under `/api/v1` with validation, a uniform success/error envelope, rate limiting on authentication routes and a health endpoint.
- An idempotent seed script through which the maintainer loads destinations and activities.
- A responsive user interface usable from a screen width of 360 px.

1.5.2 Out of scope

- Booking or payment of any kind (flights, trains, hotels, activities); the budget is an estimate, not a quotation.
- Live prices, seat or room availability, weather or traffic information.
- An administration panel; catalogue data is maintained through the seed files only.
- Native mobile applications, chat assistants, social features such as sharing or reviews, and multi-destination (multi-city) trips.
- Manual drag-and-drop editing of the generated itinerary; a saved trip can be annotated and its status changed, but the schedule itself is produced by the algorithm.

1.6 Core Components

The application follows a three-tier architecture. The presentation tier is a React single-page application that runs in the browser. The application tier is a Node.js/Express REST API organised in layers — middleware, routes and controllers, services, Mongoose models and provider adapters. The data tier is a MongoDB database. External Google services are reached only through the provider adapters, so that the rest of the system does not depend on them. Figure 1.1 shows the architecture and Table 1.1 lists the modules.

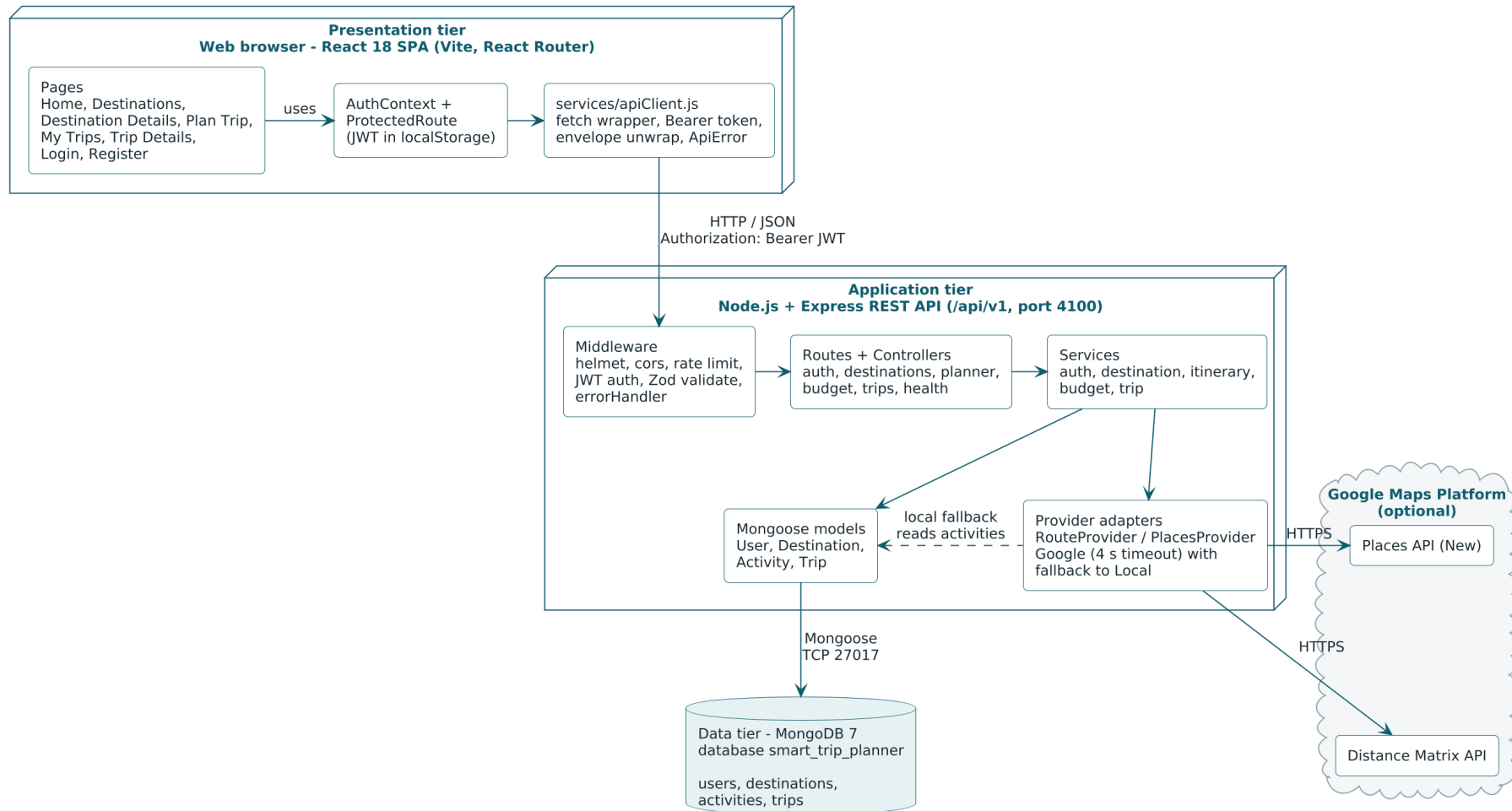


Figure 1.1: System Architecture of Smart Trip Planner

Table 1.1: Core Components (Modules) of the System

Module	Responsibility	Main elements
Authentication	Registration, login, password hashing, issue and verification of JSON Web Tokens, session restore, protection of private routes.	authService, auth middleware, /auth/* routes; AuthContext, ProtectedRoute, Login and Register pages
Destination catalogue	Storage and retrieval of destinations; keyword search, category filter, pagination; destination details.	Destination model, destinationService, /destinations routes; Home, Destinations and Destination Details pages
Activity / Places	Places to visit and things to do at a destination with coordinates, duration, fee, rating and best time; discovery of nearby places.	Activity model, PlacesProvider, /destinations/:slug/discover route
Trip Planner	Collects and validates the plan input, coordinates itinerary generation and budget estimation, returns the preview.	Plan Trip page, plannerController, /planner/preview route, Zod validators
Itinerary Generator	Scores activities, fills each day up to the capacity of the pace, orders stops by distance, schedules start and end times.	itineraryService, RouteProvider, utils/geo, time helpers
Budget Estimator	Pure function that computes rooms, nights, the four cost components, the total and the per-person amount.	budgetService, /budget/estimate route, BudgetBreakdown component
My Trips	Create, list, read, update and delete saved trips of the logged-in user with ownership isolation.	Trip model, tripService, /trips routes; My Trips and Trip Details pages
Provider adapters	Uniform interfaces for route and place data with a local implementation and a Google implementation that falls back to local on failure.	providers/ (local, google, index.js)

1.7 Development Tools and Technologies

Table 1.2 lists the technologies used. Versions are the major versions recorded in the project's `package.json` files and `docker-compose.yml`.

Table 1.2: Development Tools and Technologies

Technology	Version / Role	Reason for selection
HTML5, CSS3, JavaScript (ES modules)	Languages of the client and the server	One language across both tiers reduces context switching for a three-member team; hand-written CSS with design tokens keeps the bundle small and avoids dependence on a UI kit.
React.js	18.x — user-interface library	Component model suits the repeated UI elements (destination cards, itinerary days, budget breakdown); Context API is sufficient for the authentication state.

Technology	Version / Role	Reason for selection
Vite	8.x — development server and bundler	Fast start-up and hot reload; built-in proxy forwards <code>/api</code> to the Express server during development.
React Router	6.x — client-side routing	Declarative routes, route parameters (<code>:slug</code> , <code>:id</code>) and redirect state used by <code>ProtectedRoute</code> .
Node.js	20 or later — server runtime	Non-blocking I/O fits an API that mostly waits on the database and on external HTTP services; native <code>fetch</code> and ES modules are available.
Express.js	5.x — web framework	Minimal and well documented; the middleware pipeline maps directly to the cross-cutting concerns of the design (security headers, CORS, rate limit, validation, error handling).
MongoDB	7.x (Docker image <code>mongo:7</code>) — database	A trip is naturally one document containing embedded itinerary days, items and a budget; a document database stores it without joins.
Mongoose	9.x — object-document mapper	Schema definition, validation, indexes, timestamps and population of references.
Zod	4.x — request validation	Declarative schemas per route; validation errors are converted to a uniform <code>VALIDATION_ERROR</code> response with field-level details.
jsonwebtoken, bcryptjs	9.x, 3.x — authentication	Stateless token-based sessions suitable for a single-page application; <code>bcrypt</code> is a deliberately slow, salted password hash.
helmet, cors, express-rate-limit, morgan	Security and logging middleware	Standard HTTP hardening, origin restriction, brute-force protection on <code>/auth/*</code> and request logging in development.
Google Places API (New), Google Distance Matrix API	Optional external services	Real place search and road distances when an API key is available; isolated behind provider interfaces.
Vitest, Supertest, mongodb-memory-server	5.x, 7.x, 11.x — testing	Unit tests for the algorithms and API tests against an in-memory database, without touching development data.
ESLint	Static analysis	Consistent code style and early detection of common mistakes in both workspaces.
Docker Compose	Local infrastructure	Starts the MongoDB 7 container with one command so that every team member has the same database version.
Git	Version control	Parallel work of the backend, frontend and documentation members with history and review.
PlantUML	UML diagrams	Diagrams are kept as text sources under <code>docs/diagrams/src</code> and regenerated when the design changes.
Code editor (e.g. Visual Studio Code)	Editing and debugging	Any editor with JavaScript and ESLint support can be used; no project file depends on a particular editor.

1.8 Hardware and Software Requirements

Table 1.3: Requirements for Development

Item	Requirement
Processor	64-bit dual-core processor or better (Intel Core i3 / AMD Ryzen 3 / Apple M-series)
Memory	8 GB RAM recommended (Node.js, Vite, Docker and a browser run together)
Storage	About 2 GB free disk space for dependencies, the Docker image and the database
Operating system	Windows 10/11, macOS or a current Linux distribution
Runtime	Node.js 20 or later with npm
Database	MongoDB 7 — through Docker Desktop / Docker Compose (<code>mongo:7</code>), or a local MongoDB installation on port 27017
Tools	Git, a code editor such as Visual Studio Code, a modern web browser with developer tools
Network	Internet connection for installing packages; required at run time only when the Google providers are enabled
Optional	Google Maps Platform API key (<code>GOOGLE_MAPS_API_KEY</code>) with Places API (New) and Distance Matrix API enabled

Table 1.4: Requirements for End Users

Item	Requirement
Device	Desktop, laptop, tablet or smartphone; the layout is responsive from a screen width of 360 px
Browser	A current version of Chrome, Edge, Firefox or Safari with JavaScript and local storage enabled
Network	Connection to the server on which the application is hosted (for the demonstration: <code>http://localhost:5173</code>)
Software installation	None — the application runs entirely in the browser
Account	Not required for browsing and previewing a plan; an e-mail address and a password of at least 8 characters are required to save trips

1.9 Assumptions and Constraints

1.9.1 Assumptions

- Destination and activity data (coordinates, visit durations, entry fees, daily costs) are curated by the project team and loaded with the seed script; they are representative values, not live prices.
- All amounts are in Indian Rupees and are stored as whole rupees.

- Two travellers share one room, therefore the number of rooms is the number of travellers divided by two, rounded up.
- Food and local transport costs are per person per day; the stay cost is per room per night.
- Without the Google provider, road distance is approximated as the great-circle (haversine) distance multiplied by a road factor of 1.3, and local travel speed is taken as 25 km/h.
- Sightseeing starts at 09:00 and one 60-minute lunch break is taken per day.
- A trip concerns a single destination and lasts between 1 and 14 days for 1 to 12 travellers.

1.9.2 Constraints

- The project is a minor project of one semester developed by three members; features are limited to the scope stated in Section 1.5.
- Only free and open-source tools are used; the Google APIs are optional because they need a billing-enabled key.
- The itinerary is produced by a greedy heuristic. It is deterministic and fast but does not guarantee the mathematically shortest route.
- The catalogue is limited to the 12 seeded destinations with 8 to 10 activities each.
- Authentication uses a bearer token kept in the browser's local storage with a validity of 7 days; there is no password-reset or e-mail verification flow.
- The demonstration environment is a single machine (client on port 5173, API on port 4100, MongoDB on port 27017).

2. REQUIREMENT DETERMINATION & ANALYSIS

Requirements were determined by studying how trips are planned at present (Section 1.1), by examining the information that travel portals and blogs provide for the selected destinations, and by discussing within the team which steps of manual planning can be automated reliably. The result was written down as a system design and API contract, from which the requirements below are taken. Each functional requirement corresponds to behaviour that can be demonstrated through the user interface and verified through the REST API.

2.1 Functional Requirements

Functional requirements are grouped by module. All API paths are relative to the base path `/api/v1`; requests and responses are JSON. A successful response has the form `{ success: true, data, meta? }` and a failed response the form `{ success: false, error: { code, message, details? } }`.

2.1.1 Authentication module

Table 2.1: Functional Requirements — Authentication

ID	Requirement	Input	Output / Behaviour
FR-01	Register a new traveller	POST <code>/auth/register</code> with name (2–60 characters), e-mail, password (at least 8 characters)	Password is hashed with <code>bcrypt</code> ; user is created with role <code>traveler</code> ; response 201 with <code>{ user, token }</code> . Duplicate e-mail gives 409 <code>CONFLICT</code> ; invalid fields give 400 <code>VALIDATION_ERROR</code> with details.
FR-02	Log in	POST <code>/auth/login</code> with e-mail and password	On a match, response <code>{ user, token }</code> ; the client stores the token and returns the user to the page he or she came from. Wrong e-mail or password gives 401 <code>UNAUTHORIZED</code> without revealing which one was wrong.
FR-03	Restore session	GET <code>/auth/me</code> with header <code>Authorization: Bearer <jwt></code>	Returns <code>{ user }</code> for a valid token so that a page reload keeps the traveller logged in; an expired or invalid token gives 401 and the client logs out.
FR-04	Log out and protect private pages	Logout action; navigation to <code>/trips</code> or <code>/trips/:id</code>	Logout removes the token from local storage. A visitor who opens a private page is redirected to <code>/login</code> and, after login, back to the requested page.

2.1.2 Destination catalogue and places module

Table 2.2: Functional Requirements — Destinations and Places

ID	Requirement	Input	Output / Behaviour
FR-05	Browse destinations	GET /destinations with optional page (default 1) and limit (default 12, maximum 50)	Returns { items, total, page, limit }; the client shows a grid of destination cards with image, state, category, tagline and rating.
FR-06	Search and filter destinations	Query parameters search (keyword) and category (one of the seven categories)	Only matching destinations are returned; an empty result is shown as an explicit empty state.
FR-07	View destination details	GET /destinations/:slug	Returns { destination, activities }: overview, best season, ideal days, daily cost table for the three tiers and all activities. Unknown slug gives 404 NOT_FOUND.
FR-08	Discover nearby places	GET /destinations/:slug/discover with optional type	Returns { items } of discovered places (name, type, location, rating, address, source) and meta.provider (local or google) so that the interface can label the source.

2.1.3 Trip planner, itinerary generator and budget estimator

Table 2.3: Functional Requirements — Planning

ID	Requirement	Input	Output / Behaviour
FR-09	Accept and validate plan input	PlanInput: destinationId, startDate, endDate (YYYY-MM-DD), travelers (1–12), budgetTier, optional interests[], optional pace (default balanced)	Rules: end date not before start date; number of days (difference + 1) at most 14; start date not in the past. Violations give 400 VALIDATION_ERROR listing every failing field. The form can be pre-filled through /plan?destination=<slug>.
FR-10	Generate day-wise itinerary	POST /planner/preview with PlanInput	For each day: date, ordered items with name, type, start and end time, duration, entry fee, distance and minutes from the previous stop; totals of visit hours and travel kilometres. Days without activities are returned as free days with a note. The same input always gives the same itinerary.
FR-11	Estimate budget with the plan	Same request as FR-10	Budget with tier, currency INR, rooms, nights, break-up (stay, food, transport, activities), total and per-person amount; activity cost uses the entry fees of the scheduled items.

ID	Requirement	Input	Output / Behaviour
FR-12	Quick budget estimate	POST /budget/estimate with destinationId, days, travelers, budgetTier	Returns { budget } computed with the same formula and activity cost 0; used to compare tiers before a plan is generated.
FR-13	Preview without login and keep a pending plan	Visitor presses Save while logged out	The plan preview is public. On Save the PlanInput is kept in session storage (stp.pendingPlan), the visitor logs in or registers, is returned to /plan with the preview restored, and can then save.

2.1.4 My Trips module

Table 2.4: Functional Requirements — My Trips (login required)

ID	Requirement	Input	Output / Behaviour
FR-14	Save a trip	POST /trips with PlanInput and optional title (at most 80 characters)	The server regenerates the itinerary and the budget from the PlanInput — numbers computed by the client are never trusted — and stores the trip for the logged-in user with status planned. Response 201 { trip }. Default title: “<days> days in <destination>”.
FR-15	List my trips	GET /trips with optional status	Returns the user’s trips, newest first, as summaries: title, destination, dates, days, travellers, tier, status, total and per-person budget.
FR-16	View trip details	GET /trips/:id	Returns the complete trip with the destination populated; the page shows the day-wise itinerary, the budget planner, notes and status.
FR-17	Update a trip	PATCH /trips/:id with any of title, notes (at most 1000 characters), status (planned, completed, cancelled)	Returns the updated { trip }; other fields cannot be changed through this request.
FR-18	Delete a trip	DELETE /trips/:id	Removes the trip and returns { id }; the client returns to the My Trips list.
FR-19	Isolate trips per user	Any /trips request	Without a valid token: 401. A trip that belongs to another user is reported as 404 NOT_FOUND, so that its existence is not disclosed. A malformed identifier gives 400.

2.1.5 System and provider functions

Table 2.5: Functional Requirements — System Functions

ID	Requirement	Input	Output / Behaviour
FR-20	Report service health	GET /health	Returns { status: "ok", db: "connected", provider } where provider is local or google.
FR-21	Select provider and fall back	Environment variable GOOGLE_MAPS_API_KEY	Empty key: local providers (haversine distance, activities collection). Key present: Google providers; every call has a 4-second timeout and any failure falls back to the local provider with a logged warning.
FR-22	Return uniform errors	Any failing request	Errors are mapped centrally to the codes <code>VALIDATION_ERROR 400</code> , <code>UNAUTHORIZED 401</code> , <code>FORBIDDEN 403</code> , <code>NOT_FOUND 404</code> , <code>CONFLICT 409</code> , <code>RATE_LIMITED 429</code> and <code>INTERNAL_ERROR 500</code> ; stack traces are never sent to the client.
FR-23	Seed catalogue data	Maintainer runs <code>npm run seed</code>	Loads the 12 destinations and their activities; the script is idempotent, so running it again does not create duplicates.
FR-24	Show loading, empty and error states	Every data view of the client	A skeleton is shown while loading, an explanatory message when there is no data, and an error message with a Retry action when a request fails.

2.1.6 Non-functional requirements

Table 2.6: Non-Functional Requirements

Category	Requirement
Performance	A plan preview with the local provider involves one database read and in-memory computation over at most about ten activities and should be returned well within two seconds on the demonstration machine. External provider calls are limited by a 4-second timeout. List responses are paginated (default 12, maximum 50 items) and request bodies are limited to 100 kB.
Security	Passwords are stored only as bcrypt hashes (cost factor 10) and the hash is excluded from every query result and JSON output. Sessions use signed JSON Web Tokens that expire after 7 days. <code>helmet</code> sets secure HTTP headers and CORS accepts only the configured client origin. Authentication routes are rate-limited to 20 requests per 15 minutes per IP address. Every request body, query and path parameter is validated with <code>Zod</code> . Trips are isolated per owner, and secrets are read from an environment file that is not committed.
Usability	Responsive layout from 360 px; forms are keyboard accessible with visible focus and labelled inputs; the <code>prefers-reduced-motion</code> setting is respected; amounts are formatted in the Indian numbering system (<code>en-IN</code> , <code>INR</code> , no decimals); every view has explicit loading, empty and error states.

Category	Requirement
Reliability	The application works without any external service. When Google providers are enabled, a timeout or upstream error causes an automatic fallback to the local provider. Itinerary generation is deterministic (ties are broken by activity name). Fonts are self-hosted so that the demonstration works offline.
Maintainability	Layered server structure (routes, controllers, services, models, providers) with business logic kept out of controllers; the budget estimator is a pure function; configuration is read and validated once; ESLint is used in both workspaces; unit, API and provider tests run with Vitest, Supertest and an in-memory MongoDB; the design document is the single source of truth.
Portability	Runs on Windows, macOS and Linux with Node.js 20 or later; MongoDB is provided as a Docker container; the client runs in any current browser; all environment-specific values are set through environment variables.

2.2 Targeted Users

The system distinguishes two kinds of interactive users by authentication state — the guest and the registered traveller. The groups below describe who these users typically are and what they need from the system. The maintainer is not an interactive role in the user interface.

Table 2.7: Targeted Users

User group	Description and needs	Access
Guest (visitor)	Anyone who opens the site without logging in: wants to explore destinations, compare costs and try out a plan before deciding to create an account.	Browse, search, view details, discover places, generate and preview a plan, register, log in
Registered traveller	A visitor who has created an account: wants to keep plans, return to them before and during the trip, add notes and mark trips as completed or cancelled.	Everything a guest can do, plus save, list, view, update and delete own trips
Families and groups	Travel with 3 to 12 people: need the number of rooms, the group total and the per-person share, and usually prefer a relaxed or balanced pace.	As guest or registered traveller
Students and budget travellers	Cost-sensitive users: compare the economy tier with the standard tier, prefer free or low-fee activities and want to see the full cost before committing.	As guest or registered traveller
Maintainer (administrator)	A project team member who curates destinations and activities and loads them with the seed script. The <code>admin</code> role exists in the user schema for future use; no administration screens are part of this project.	Command line and database only

3. SYSTEM DESIGN

The design is documented with UML 2 diagrams. The use case diagram gives the functional view, activity diagrams describe the main work flows, sequence (interaction) diagrams show how the objects of the three tiers collaborate, and the class diagram gives the static structure. The data dictionary at the end of the chapter defines every stored field.

3.1 Use Case Diagram

Figure 3.1 shows the use cases of Smart Trip Planner inside the system boundary. The primary actors are the Guest and the Registered Traveller. The Registered Traveller is a specialisation of the Guest and therefore inherits all guest use cases. The two Google services are secondary actors: they do not start any use case but take part in one when the Google providers are enabled.

The relationships between use cases in the diagram are read as follows. **Plan trip** always includes **Generate itinerary** and **Estimate budget**; the include arrows point from the base use case to the included ones. **Save trip** extends **Plan trip** at the point where the plan preview is shown, under the condition that the traveller is logged in; the extend arrow points from the extending use case to the base use case. If the traveller is not logged in, the client redirects to Login and restores the pending plan afterwards. **Search / filter destinations** extends **Browse destinations**, because the list is also usable without any filter.

Table 3.1: Actors

Actor	Type	Description
Guest (Visitor)	Primary	A person using the application without being logged in. Can browse, search, view details, discover places, plan a trip, register and log in.
Registered Traveller	Primary	A logged-in user (role <code>traveler</code>). Inherits the guest use cases and can additionally save a trip and manage saved trips.
Google Places API	Secondary (external system)	Supplies tourist attractions for “Discover nearby places” when an API key is configured; otherwise the local activities are used.
Google Distance Matrix API	Secondary (external system)	Supplies road distance and travel time between two stops for “Generate itinerary” when an API key is configured; otherwise the local haversine estimate is used.

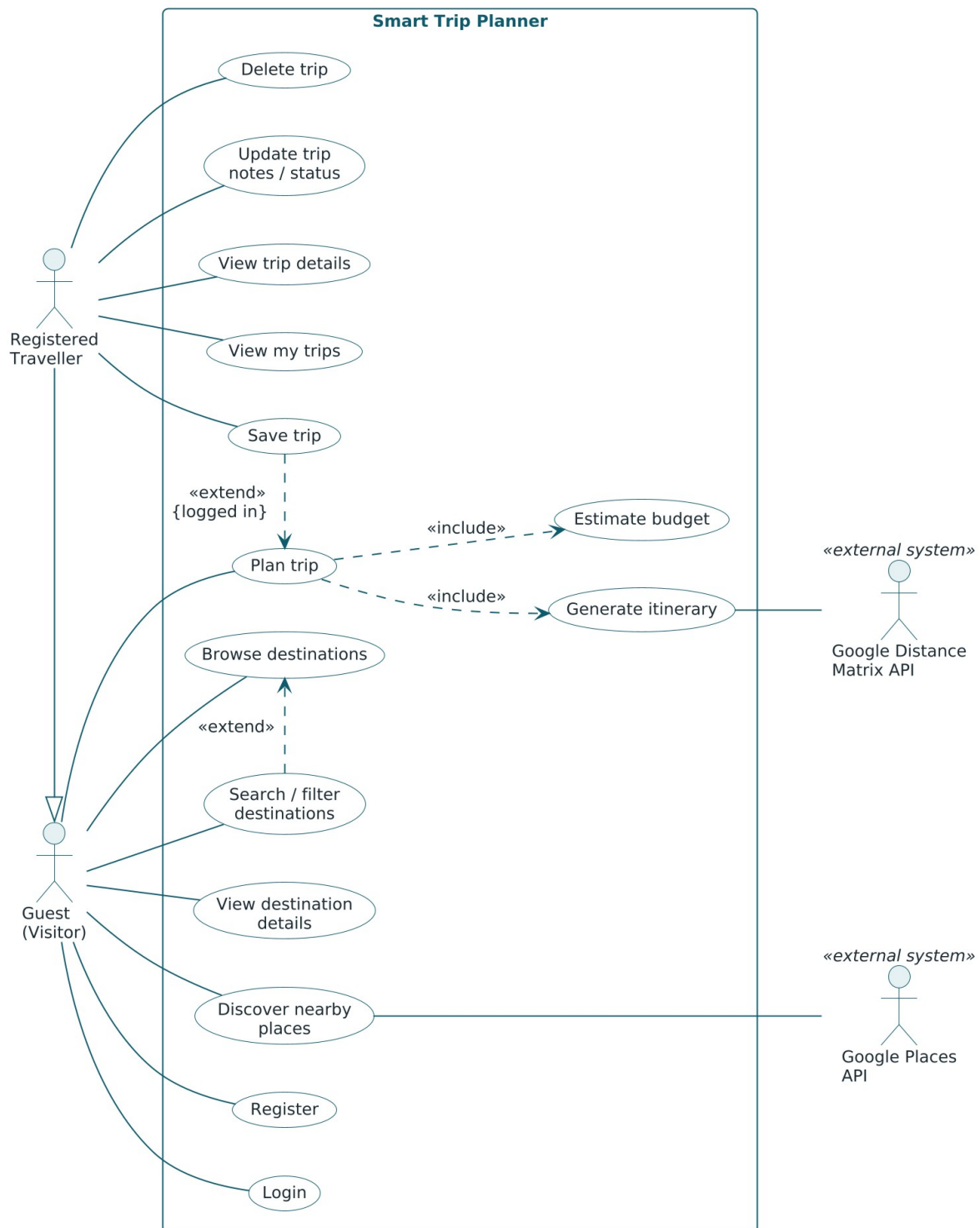


Figure 3.1: Use Case Diagram

3.1.1 Use case descriptions

Table 3.2: Use Case Description — Plan Trip

Item	Description
Use case ID	UC-05
Name	Plan trip (includes Generate itinerary and Estimate budget)

Item	Description
Actors	Guest or Registered Traveller (primary); Google Distance Matrix API (secondary, optional)
Preconditions	Destinations and activities have been seeded. The actor has opened the Plan Trip page, optionally with a destination pre-selected.
Main flow	1. The actor selects a destination and enters start date, end date, number of travellers, budget tier, interests and pace. 2. The client validates the form and sends <code>POST /planner/preview</code>. 3. The server validates the PlanInput and loads the destination with its activities. 4. The system generates the itinerary (included use case Generate itinerary). 5. The system estimates the budget (included use case Estimate budget). 6. The server returns destination, days, itinerary, budget and the provider used. 7. The client displays the day-wise itinerary and the budget breakdown.
Alternate flows	2a. A field is missing or invalid: the client shows the error beside the field and nothing is sent. 3a. The end date is before the start date, the trip is longer than 14 days or the start date is in the past: the server answers <code>400 VALIDATION_ERROR</code> and the client shows the messages. 3b. The destination does not exist: <code>404 NOT_FOUND</code>. 4a. The Google provider times out or fails: distances are taken from the local provider and the response is labelled <code>provider = local</code>. 4b. There are fewer activities than days: remaining days are returned as free days.
Postconditions	A plan preview is displayed. Nothing is stored on the server. The actor may continue with Save trip (UC-10).

Table 3.3: Use Case Description — Save Trip

Item	Description
Use case ID	UC-10
Name	Save trip (extends Plan trip)
Actors	Registered Traveller
Preconditions	A plan preview is displayed (extension point of UC-05). The actor is logged in, i.e. the client holds a valid token.
Main flow	1. The actor presses Save, optionally after editing the title. 2. The client sends <code>POST /trips</code> with the PlanInput, the title and the bearer token. 3. The authentication middleware verifies the token and identifies the user. 4. The server validates the PlanInput and regenerates the itinerary and the budget. 5. The server stores the trip with the user as owner and status <code>planned</code> and answers <code>201</code> with the trip. 6. The client opens the Trip Details page of the new trip.

Item	Description
Alternate flows	1a. The actor is not logged in: the client keeps the PlanInput in session storage, redirects to Login (or Register), returns to the Plan Trip page with the preview restored, and the flow continues at step 1. 3a. The token is missing, invalid or expired: 401 UNAUTHORIZED; the client logs out and redirects to Login. 4a. The PlanInput is no longer valid (for example the start date has passed): 400 VALIDATION_ERROR.
Postconditions	A new document exists in the <code>trips</code> collection, owned by the actor; it appears first in My Trips.

Table 3.4: Use Case Description — Login

Item	Description
Use case ID	UC-09
Name	Login
Actors	Guest
Preconditions	The guest has registered earlier (UC-08) and is not logged in.
Main flow	1. The guest opens the Login page directly or is redirected to it from a private page or from Save trip. 2. The guest enters e-mail and password and submits the form. 3. The client sends <code>POST /auth/login</code>. 4. The server applies the rate limit, validates the body, finds the user by e-mail and compares the password with the stored bcrypt hash. 5. The server signs a JSON Web Token valid for 7 days and returns <code>{ user, token }</code>. 6. The client stores the token in local storage, sets the authenticated user and navigates to the page the guest came from (default: My Trips).
Alternate flows	4a. E-mail unknown or password wrong: 401 UNAUTHORIZED; the client shows “Invalid email or password”. 4b. More than 20 authentication requests in 15 minutes from one IP address: 429 RATE_LIMITED. 4c. Malformed e-mail or empty password: 400 VALIDATION_ERROR with field details.
Postconditions	The actor is a Registered Traveller for this session; after a page reload the session is restored through <code>GET /auth/me</code> .

Table 3.5: Use Case Description — Discover Nearby Places

Item	Description
Use case ID	UC-04
Name	Discover nearby places
Actors	Guest or Registered Traveller (primary); Google Places API (secondary, optional)
Preconditions	The actor is viewing the details page of a destination.

Item	Description
Main flow	1. The actor opens the “Discover nearby” section and optionally chooses a type of place. 2. The client sends <code>GET /destinations/:slug/discover?type=<type></code>. 3. The server asks the configured PlacesProvider for places of that destination. 4. With the Google provider, a text search for tourist attractions in “<name>, <state>” is sent to the Places API and the results are mapped to the common place format. 5. The server returns the list and <code>meta.provider</code>. 6. The client lists the places with name, type, rating and address and labels the source.
Alternate flows	3a. No API key is configured: the local provider returns the matching records of the <code>activities</code> collection. 4a. The Google call fails or exceeds 4 seconds: the local provider is used and a warning is logged. 5a. No place matches the type: an empty list is returned and the client shows the empty state.
Postconditions	The actor sees places of the destination; no data is changed.

3.2 Activity Diagram

The central work flow of the application — from choosing a destination to opening the saved trip — is modelled with swimlanes for the Traveller, the React client, the Express API and the data/provider layer. For legibility on an A4 page the flow is drawn in two parts: Figure 3.2 ends when the plan preview is shown, and Figure 3.3 continues from that point with saving the trip.

3.2.1 Plan trip and save trip

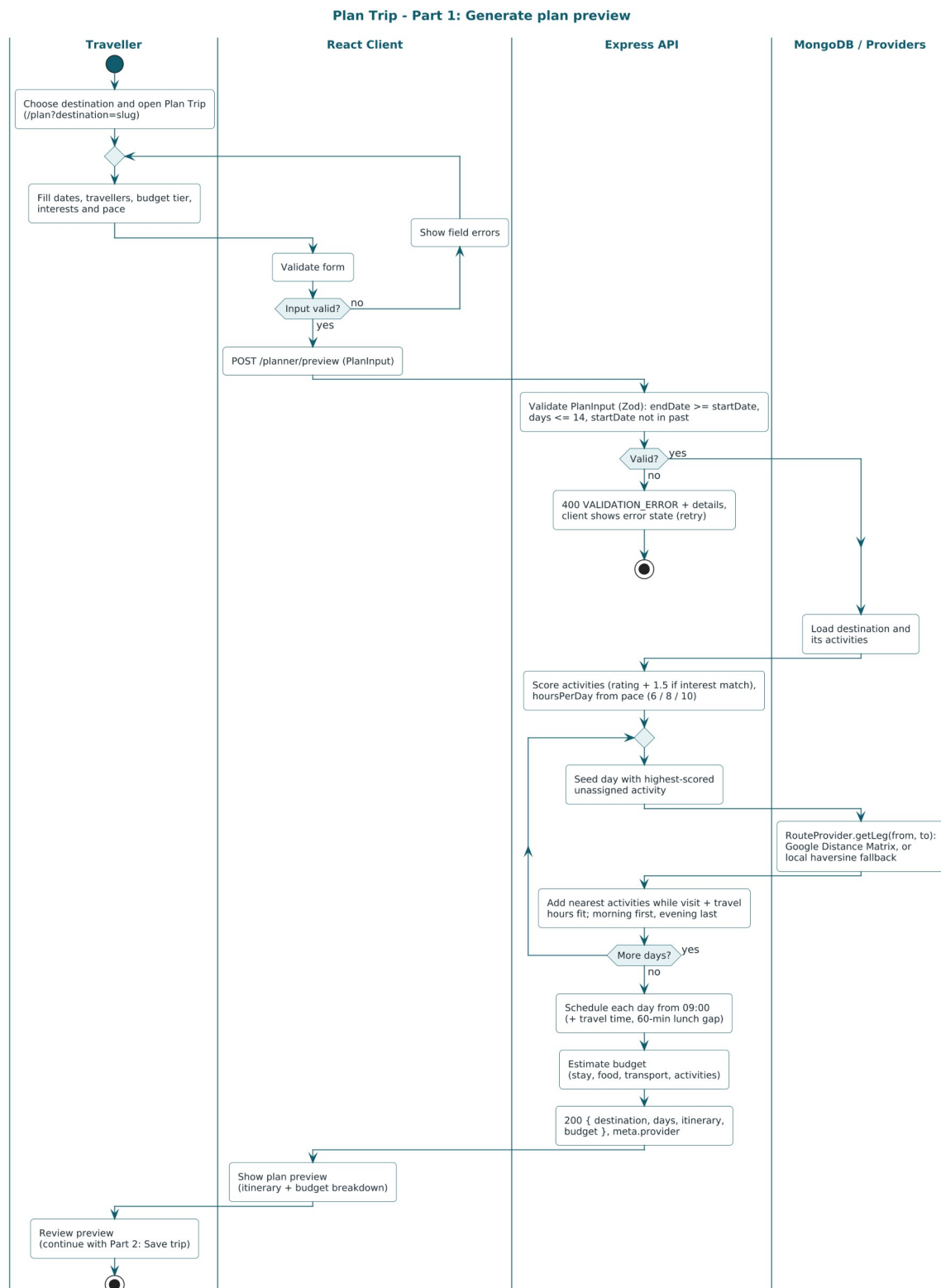


Figure 3.2: Activity Diagram — Plan Trip, Part 1 (Generate Plan Preview)

Part 1 contains two decisions on input validity. The first is taken in the client, which returns to the form with field errors; the second is taken by the API, because the server never relies on

client-side checks. The loop “More days?” represents step 3 of the itinerary algorithm described in Section 3.2.3.

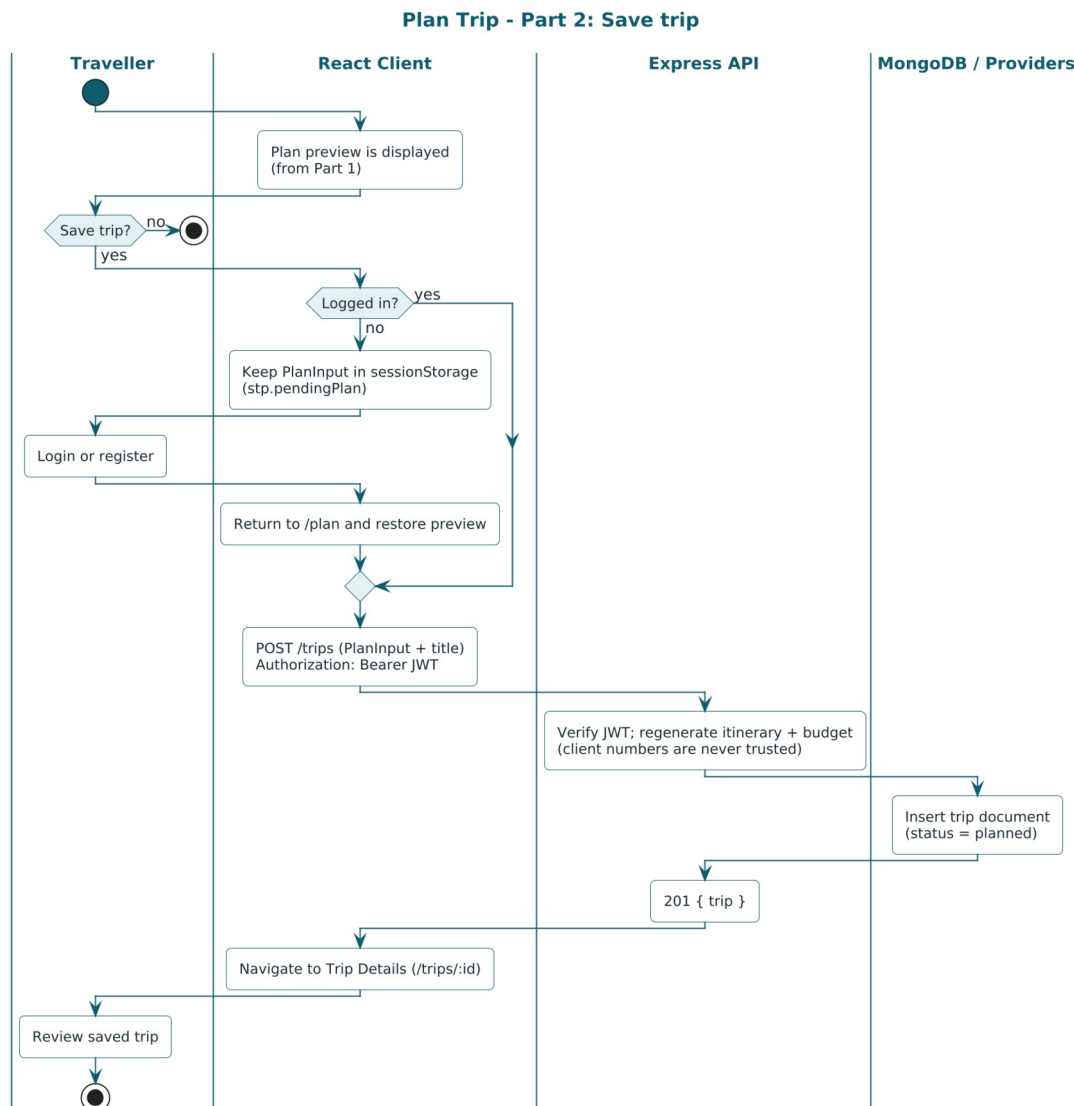


Figure 3.3: Activity Diagram — Plan Trip, Part 2 (Save Trip)

Part 2 shows the decision “Logged in?”. On the negative branch the plan input is kept in session storage, the traveller logs in or registers, and the preview is restored, after which both branches merge at the request `POST /trips`. The API regenerates the plan before inserting the trip.

3.2.2 Registration and login

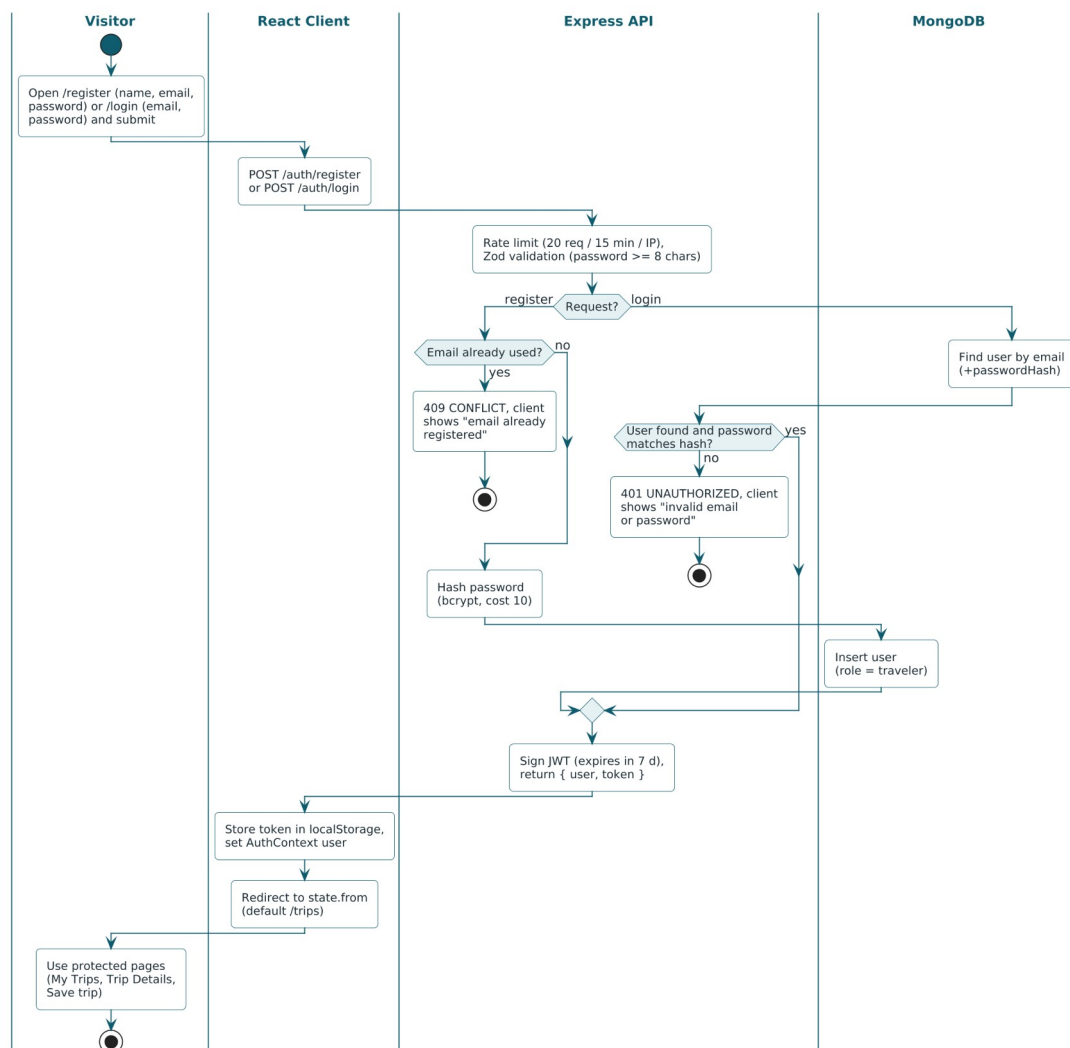


Figure 3.4: Activity Diagram — Registration and Login

Both requests pass the rate limiter and the Zod validation. Registration fails with 409 when the e-mail is already used; login fails with 401 when the user is not found or the password does not match the hash. Both successful branches merge into signing the token, which the client stores before redirecting the visitor.

3.2.3 Itinerary generation algorithm

The itinerary is generated by `itineraryService` from the activities of the destination, the number of days, the pace, the interests, the start date and a `RouteProvider`. The algorithm is deterministic: the same input always produces the same output, and ties are broken by activity name in ascending order.

1. **Score.** For every activity compute $\text{score} = \text{rating} + 1.5$ if its type is among the traveller's interests, otherwise $\text{score} = \text{rating}$.

2. **Capacity.** Set the number of usable hours per day from the pace: relaxed = 6, balanced = 8, packed = 10.
3. **Build each day** (day 1 to day N) from the pool of activities that are not yet assigned:
(a) seed the day with the highest-scored unassigned activity; (b) repeatedly take, among the top-scored half of the remaining pool, the activity that is nearest to the last stop according to the RouteProvider, and add it if visit hours plus travel hours of the day stay within the capacity — stop when nothing fits; (c) move activities whose best time is `evening` to the end of the day and those whose best time is `morning` to the front (stable reordering), then recompute the distance of every leg.
4. **Schedule.** Start each day at 09:00. The start time of a stop is the end time of the previous stop plus the travel minutes, rounded up to 5 minutes; a 60-minute lunch gap is inserted before the first item that would start after 13:00.
5. **Free days.** A day for which no activity is left receives an empty item list and the note “Free day — explore at your own pace”.

The RouteProvider returns the distance in kilometres and the travel time in minutes for one leg. The local implementation uses the haversine distance multiplied by 1.3 and an average speed of 25 km/h; the Google implementation calls the Distance Matrix API and falls back to the local one on any failure. Restricting step 3(b) to the top-scored half keeps good activities from being displaced by places that are merely close.

3.3 Interaction Diagram

Sequence diagrams show the messages exchanged between the traveller, the client objects (page, authentication context, API client), the server objects (middleware, controllers, services, providers) and the database. Solid arrows are calls, dashed arrows are returns.

3.3.1 Generate plan preview

In Figure 3.5 the `loop` fragment corresponds to the distance queries made while the days are being built. Inside it, the `alt` fragment shows the provider behaviour: when a Google key is configured and the Distance Matrix API answers within 4 seconds its values are used; in every other case the local haversine estimate is returned and the response is labelled with `provider = local`.

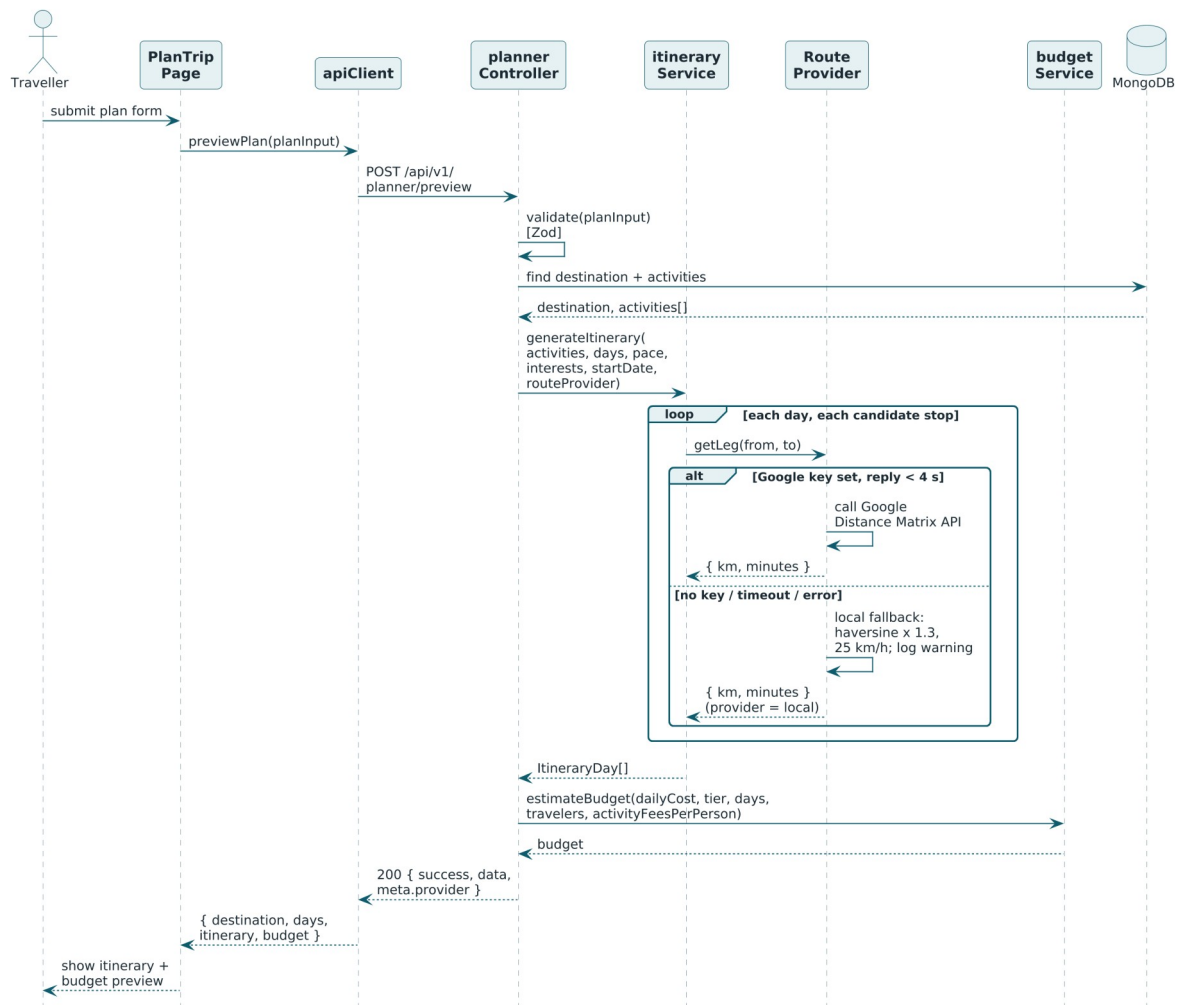


Figure 3.5: Sequence Diagram — Generate Plan Preview

3.3.2 Save trip

In Figure 3.6 the `opt` fragment is executed only when the traveller is not logged in. The request to `/trips` carries the token in the `Authorization` header; the authentication middleware verifies it before the controller is reached. The trip service then loads the destination, regenerates the itinerary and the budget on the server and inserts the trip.

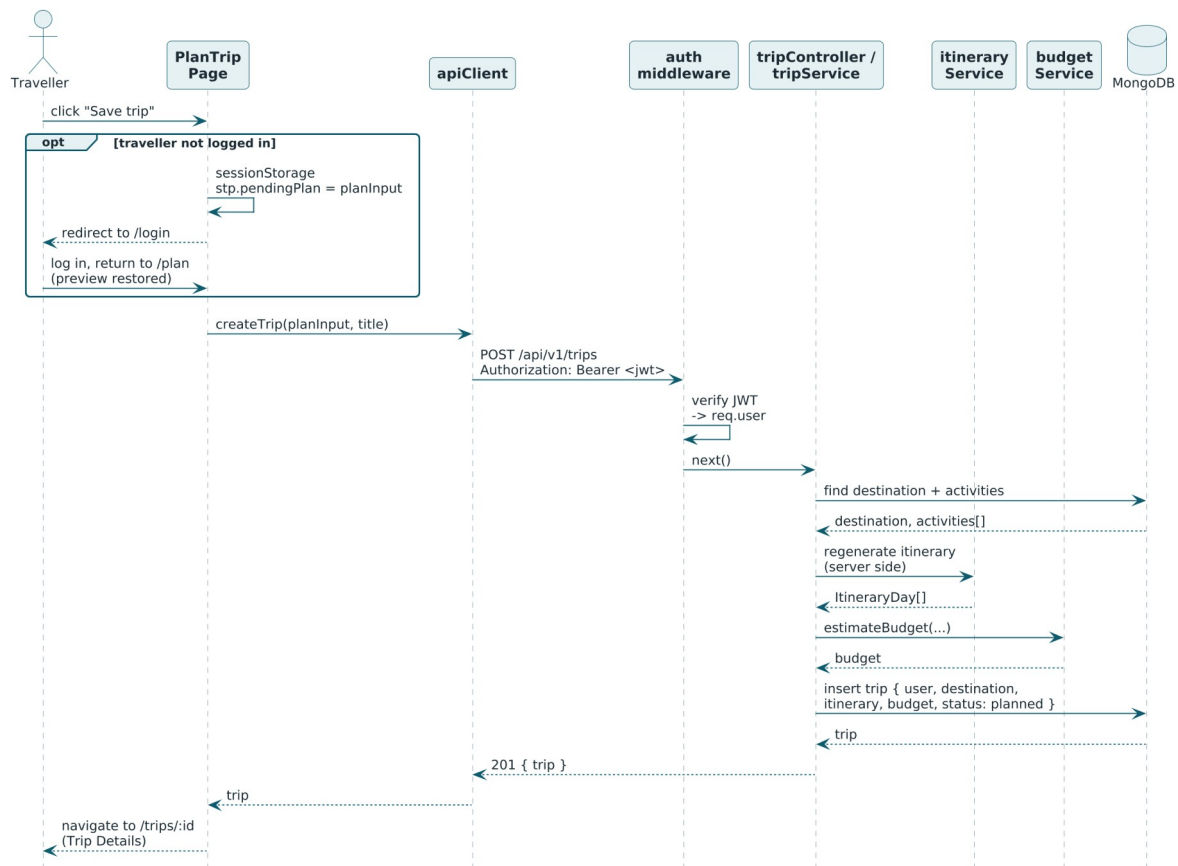


Figure 3.6: Sequence Diagram — Save Trip with JWT Authentication

3.3.3 Login and session restore

The upper part of Figure 3.7 shows both outcomes of a login attempt. The lower part shows what happens when the application is opened or reloaded: if a token is present in local storage, the authentication context calls `/auth/me`; a valid token restores the user, while an expired or invalid token leads to logout and a redirect to the Login page.

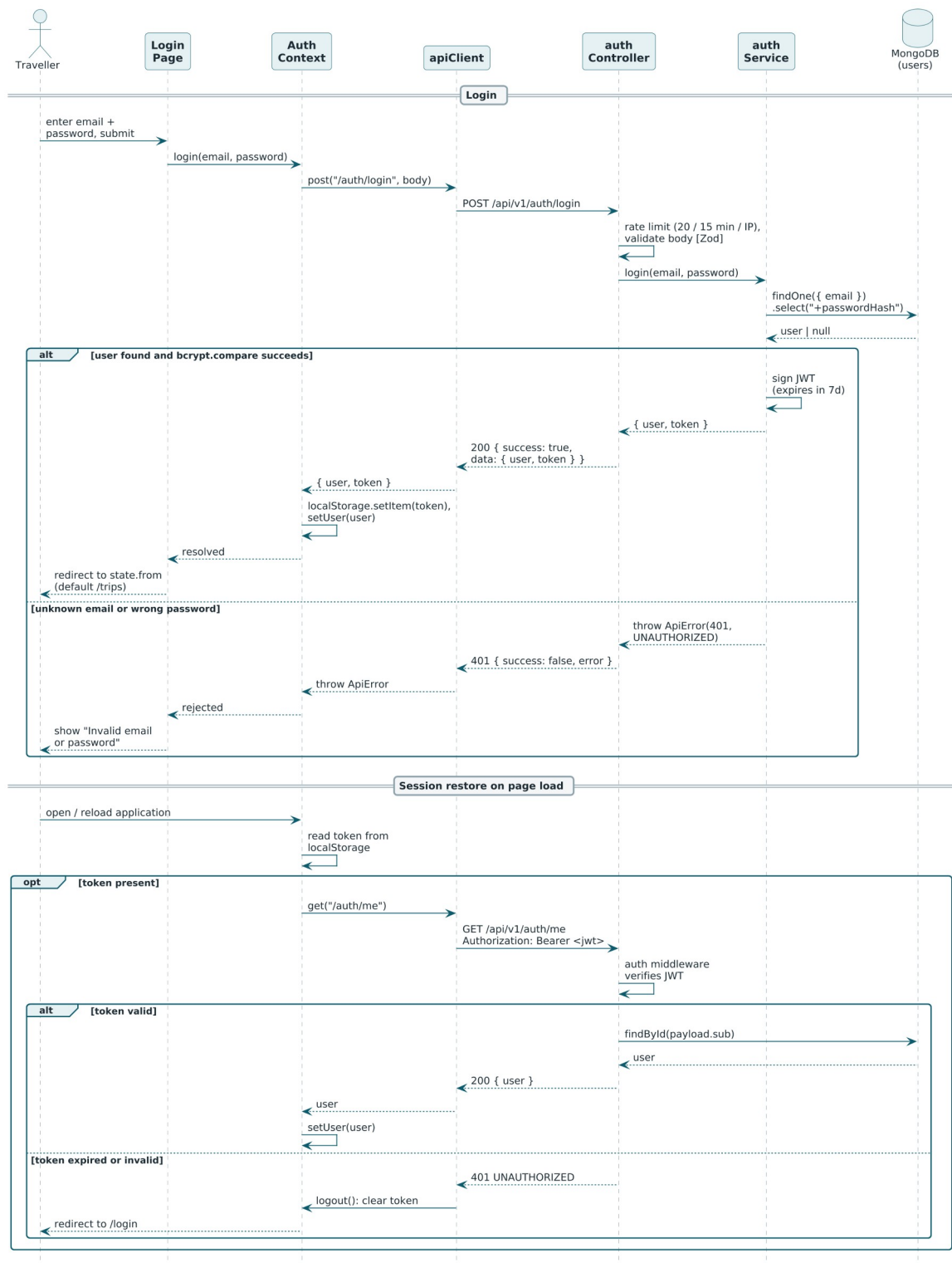


Figure 3.7: Sequence Diagram — Login and Session Restore

3.4 Class Diagram

The static structure is shown in two diagrams, and Table 3.6 summarises the responsibility of every class. Figure 3.8 contains the domain classes, which correspond to the four MongoDB

collections and their embedded structures. Figure 3.9 contains the service classes of the application tier and the provider interfaces with their realisations.

A User owns zero or more Trips and every Trip belongs to exactly one User. Many Trips can be planned for one Destination, and a Destination offers many Activities. Filled diamonds denote composition, i.e. embedded sub-documents that cannot exist without their parent: a Destination is composed of one DailyCost with three TierCost values (economy, standard, luxury) and one GeoPoint; a Trip is composed of 1 to 14 ItineraryDays, each composed of zero or more ItineraryItems, and of exactly one Budget. An ItineraryItem is a snapshot of an Activity: it copies name, type, location, duration and fee and keeps only a reference to the original, so that a saved trip stays unchanged if the catalogue is edited later.

Table 3.6: Class Responsibilities

Class	Kind	Responsibility
User	Collection	Account of a traveller or maintainer: name, unique e-mail, password hash, role.
Destination	Collection	A place that can be planned for: descriptive data, location, best season, ideal days, daily costs per tier, rating, tags.
DailyCost / TierCost	Embedded	Daily stay, food and transport cost for each of the three budget tiers of a destination.
Activity	Collection	A place to visit or thing to do at a destination with type, location, duration, entry fee, rating, best time and source.
Trip	Collection	A saved plan of one user for one destination: plan input, generated itinerary, budget, status and notes.
ItineraryDay	Embedded	One day of a trip: day number, date, ordered items, total visit hours, total travel kilometres, note.
ItineraryItem	Embedded	One scheduled stop: snapshot of the activity with start and end time and the travel leg from the previous stop.
Budget	Embedded	Result of the budget estimation: tier, currency, rooms, nights, break-up, total, per-person amount.
AuthService	Service	Registers users, verifies credentials, signs tokens and returns the current user.
DestinationService	Service	Lists, searches and filters destinations, returns details with activities and delegates discovery to the PlacesProvider.
ItineraryService	Service	Implements the scoring, day-building and scheduling algorithm of Section 3.2.3 using a RouteProvider.
BudgetService	Service	Pure function <code>estimateBudget</code> implementing the formula of Section 3.5.9.
TripService	Service	Builds a plan from a PlanInput (used for preview and save) and performs owner-restricted create, list, read, update and delete of trips.

Class	Kind	Responsibility
RouteProvider	Interface	getLeg(from, to) returns kilometres and minutes between two points. Realised by LocalRouteProvider (haversine $\times$ 1.3, 25 km/h) and GoogleRouteProvider (Distance Matrix API, 4 s timeout, fallback to local).
PlacesProvider	Interface	discover(destination, type) returns places of a destination. Realised by LocalPlacesProvider (activities collection) and GooglePlacesProvider (Places API text search, 4 s timeout, fallback to local).

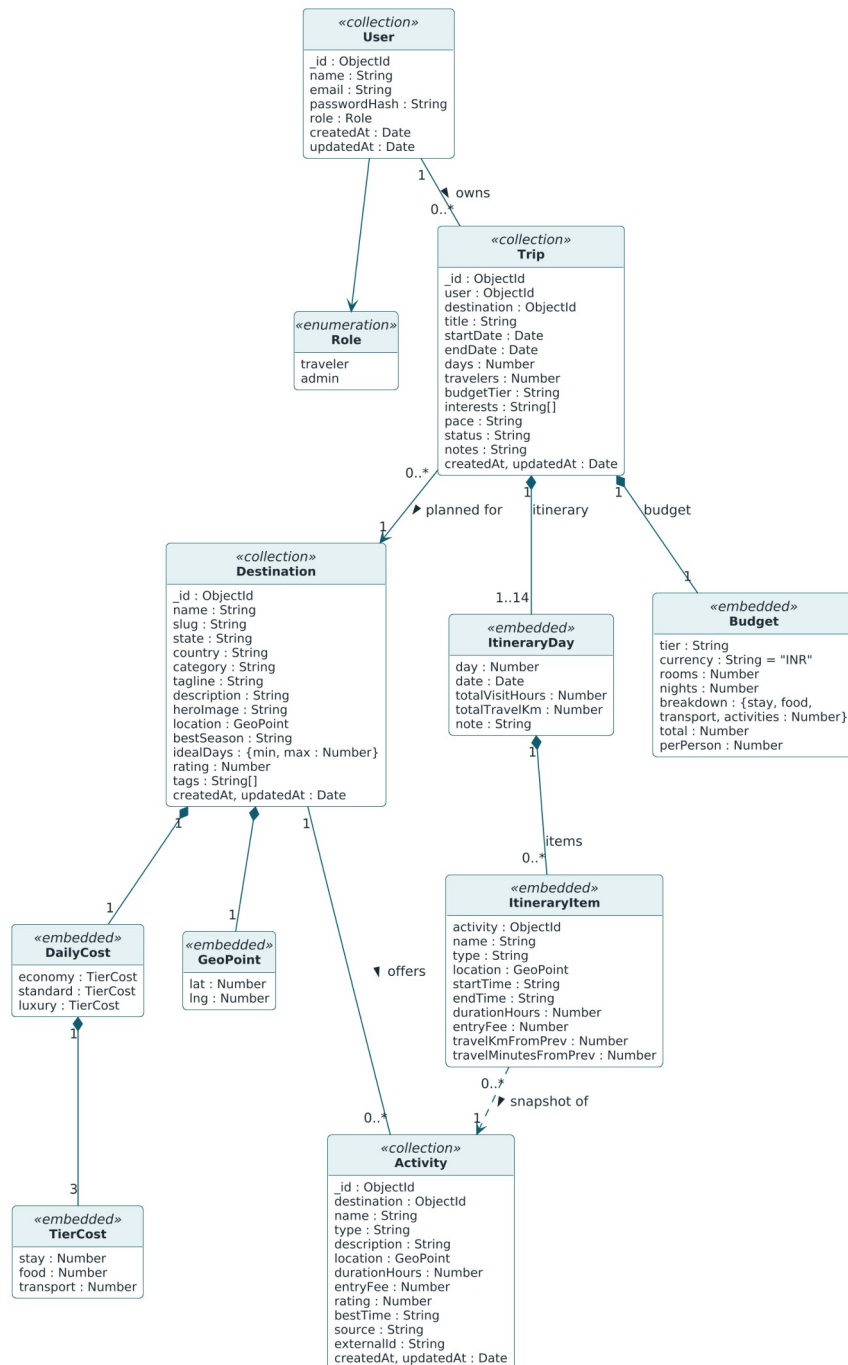


Figure 3.8: Class Diagram — Domain Model

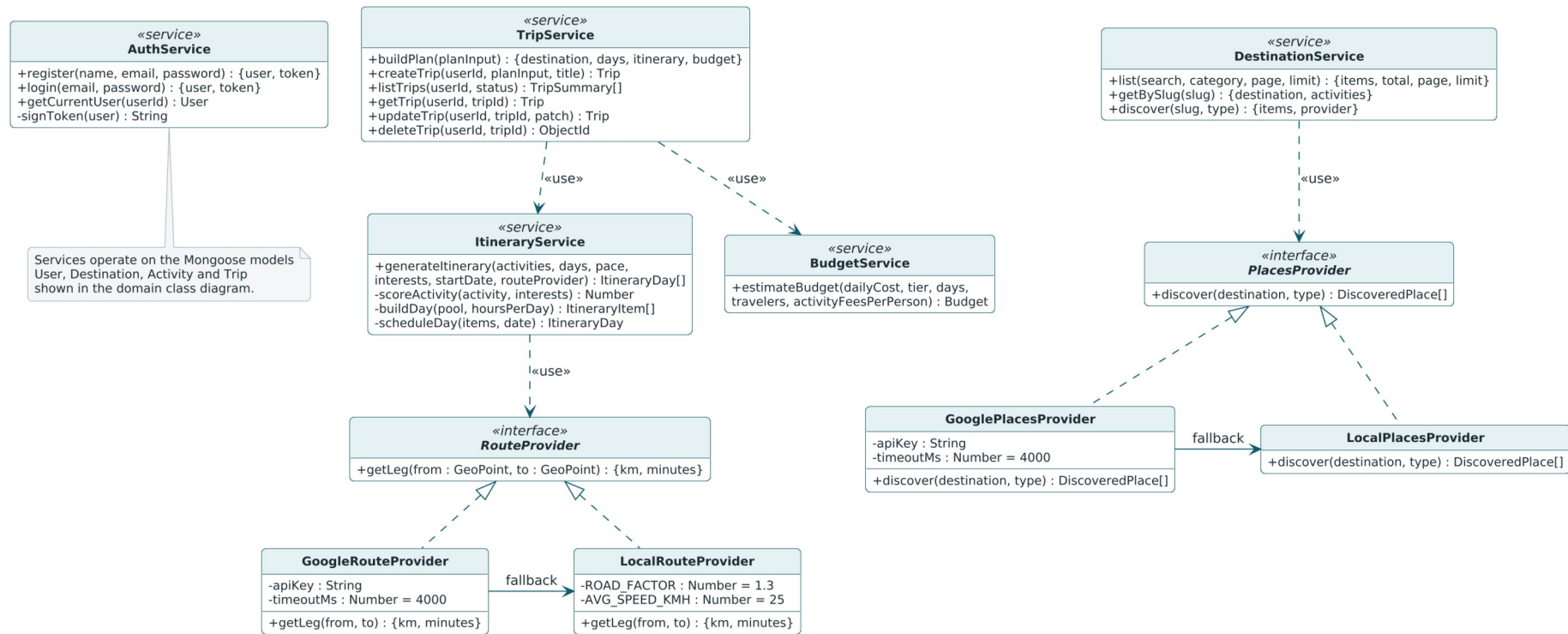


Figure 3.9: Class Diagram — Services and Provider Interfaces

3.5 Data Dictionary

The database `smart_trip_planner` contains four collections: `users`, `destinations`, `activities` and `trips`. Every document has the primary key `_id` of type `ObjectId` generated by MongoDB, and the fields `createdAt` and `updatedAt` maintained by Mongoose. References between collections are stored as `ObjectId` values and are listed as foreign keys (FK). Money is stored in Indian Rupees as whole numbers. In the column Size / Range, “variable” means that the design does not impose a length limit. In JSON responses `_id` is returned as `id`, and the internal version key is removed.

3.5.1 Collection: users

Table 3.7: Data Dictionary — users

Field	Data Type	Size / Range	Constraints	Description
<code>_id</code>	<code>ObjectId</code>	12 bytes (24 hexadecimal characters)	Primary key; generated automatically	Unique identifier of the user
<code>name</code>	<code>String</code>	2–60 characters	Required; trimmed	Display name
<code>email</code>	<code>String</code>	Variable; valid e-mail format	Required; unique; stored in lower case	Login identifier
<code>passwordHash</code>	<code>String</code>	60 characters (bcrypt output)	Required; <code>select: false</code> — never returned by queries or in JSON	bcrypt hash of the password (cost 10); the plain password must have at least 8 characters
<code>role</code>	<code>String</code>	Enumeration	Enum: <code>traveler</code> , <code>admin</code> ; default <code>traveler</code>	Authorisation role
<code>createdAt</code>	<code>Date</code>	ISO 8601 timestamp	Set automatically on insert (Mongoose timestamps)	Date and time the document was created
<code>updatedAt</code>	<code>Date</code>	ISO 8601 timestamp	Set automatically on every update (Mongoose timestamps)	Date and time of the last modification

3.5.2 Collection: destinations

Table 3.8: Data Dictionary — destinations

Field	Data Type	Size / Range	Constraints	Description
<code>_id</code>	<code>ObjectId</code>	12 bytes	Primary key; generated automatically	Unique identifier of the destination
<code>name</code>	<code>String</code>	Variable	Required	Name of the destination, e.g. “Manali”

Field	Data Type	Size / Range	Constraints	Description
slug	String	Variable	Required; unique; lower case	Identifier used in URLs, e.g. manali
state	String	Variable	Required	Indian state or union territory
country	String	Variable	Default India	Country
category	String	Enumeration	Enum: hill-station, beach, heritage, spiritual, nature, desert, adventure	Primary category used by the filter
tagline	String	At most 90 characters	Required	One-line text shown on destination cards
description	String	Variable (2–4 sentences)	Required	Overview of the destination
heroImage	String	Variable (path)	Required	Image path /images/destinations/ /<slug>.jpg, served by the client
location.lat	Number	Decimal degrees	Required	Latitude of the town centre
location.lng	Number	Decimal degrees	Required	Longitude of the town centre
bestSeason	String	Variable	Required	Best time to visit, e.g. “October – February”
idealDays.min	Number	Integer	Required	Shortest suggested trip length in days
idealDays.max	Number	Integer	Required	Longest suggested trip length in days
dailyCost	Object	Three tiers	Required; keys <i>economy</i> , <i>standard</i> , <i>luxury</i> , each of the structure in Table 3.9	Daily cost per budget tier
rating	Number	0–5	Minimum 0; maximum 5	Editorial rating
tags	Array of String	Variable	Optional	Keywords used by the search
createdAt	Date	ISO 8601 timestamp	Set automatically on insert (Mongoose timestamps)	Date and time the document was created
updatedAt	Date	ISO 8601 timestamp	Set automatically on every update (Mongoose timestamps)	Date and time of the last modification

Table 3.9: Data Dictionary — dailyCost tier (embedded in destinations)

Field	Data Type	Size / Range	Constraints	Description
stay	Number	Integer ≥ 0 (INR)	Required	Cost of one room for one night

Field	Data Type	Size / Range	Constraints	Description
food	Number	Integer ≥ 0 (INR)	Required	Food cost per person per day
transport	Number	Integer ≥ 0 (INR)	Required	Local transport cost per person per day

3.5.3 Collection: activities

Table 3.10: Data Dictionary — activities

Field	Data Type	Size / Range	Constraints	Description
_id	ObjectId	12 bytes	Primary key; generated automatically	Unique identifier of the activity
destination	ObjectId	12 bytes	Foreign key $\rightarrow$ destinations._id; required; indexed	Destination to which the activity belongs
name	String	Variable	Required; unique together with destination (compound index)	Name of the place or activity, e.g. “Solang Valley”
type	String	Enumeration	Enum: sightseeing, adventure, culture, nature, spiritual, food, shopping	Type used for matching the traveller’s interests
description	String	Variable (1–2 sentences)	Required	Short description
location.lat	Number	Decimal degrees	Required	Latitude of the place
location.lng	Number	Decimal degrees	Required	Longitude of the place
durationHours	Number	0.5–8	Required; minimum 0.5; maximum 8	Typical visit time in hours
entryFee	Number	Integer ≥ 0 (INR)	Default 0	Entry fee per person
rating	Number	0–5	Minimum 0; maximum 5	Rating used in the score
bestTime	String	Enumeration	Enum: morning, afternoon, evening, any; default any	Scheduling hint for the order within a day
source	String	Enumeration	Enum: seed, google; default seed	Origin of the record
externalId	String	Variable	Optional	Google place identifier when source is google
createdAt	Date	ISO 8601 timestamp	Set automatically on insert (Mongoose timestamps)	Date and time the document was created
updatedAt	Date	ISO 8601 timestamp	Set automatically on every update (Mongoose timestamps)	Date and time of the last modification

3.5.4 Collection: trips

Table 3.11: Data Dictionary — trips

Field	Data Type	Size / Range	Constraints	Description
_id	ObjectId	12 bytes	Primary key; generated automatically	Unique identifier of the trip
user	ObjectId	12 bytes	Foreign key → users._id; required; indexed	Owner of the trip
destination	ObjectId	12 bytes	Foreign key → destinations._id; required	Destination of the trip
title	String	At most 80 characters	Required; default “<days> days in <destination>”	Title shown in My Trips
startDate	Date	Calendar date	Required; not in the past when the trip is created	First day of the trip
endDate	Date	Calendar date	Required; not before startDate	Last day of the trip
days	Number	1–14	Integer; equals the date difference + 1	Inclusive number of days
travelers	Number	1–12	Integer	Number of travellers
budgetTier	String	Enumeration	Enum: economy, standard, luxury	Selected budget tier
interests	Array of String	0–7 values	Each value from the activity type enumeration	Interests used for scoring
pace	String	Enumeration	Enum: relaxed, balanced, packed; default balanced	Determines the hours per day (6, 8, 10)
itinerary	Array of ItineraryDay	One element per day	Embedded; structure in Table 3.12	Generated day-wise plan
budget	Budget	One object	Embedded; structure in Table 3.14	Generated budget estimate
status	String	Enumeration	Enum: planned, completed, cancelled; default planned	State of the trip
notes	String	At most 1000 characters	Default empty string	Free text added by the traveller
createdAt	Date	ISO 8601 timestamp	Set automatically on insert (Mongoose timestamps)	Date and time the document was created
updatedAt	Date	ISO 8601 timestamp	Set automatically on every update (Mongoose timestamps)	Date and time of the last modification

3.5.5 Embedded structure: itineraryDay

Table 3.12: Data Dictionary — itineraryDay (embedded in trips.itinerary)

Field	Data Type	Size / Range	Constraints	Description
day	Number	1 to days	Integer	Sequence number of the day
date	Date	Calendar date	<code>startDate + (day - 1)</code>	Calendar date of the day
items	Array of ItineraryItem	0 or more	Embedded; structure in Table 3.13; empty on a free day	Ordered stops of the day
totalVisitHours	Number	≥ 0	Derived: sum of <code>durationHours</code> of the items	Time spent at the stops
totalTravelKm	Number	≥ 0	Derived: sum of <code>travelKmFromPrev</code> of the items	Distance travelled during the day
note	String	Variable	Optional	Remark, e.g. “Free day — explore at your own pace”

3.5.6 Embedded structure: itineraryItem

An itinerary item is a snapshot of the activity at the time the plan was generated, so that a saved trip remains stable if the activity is changed or removed later.

Table 3.13: Data Dictionary — itineraryItem (embedded in itineraryDay.items)

Field	Data Type	Size / Range	Constraints	Description
activity	ObjectId	12 bytes	Foreign key → <code>activities._id</code> (reference only)	Activity from which the snapshot was taken
name	String	Variable	Copied from the activity	Name of the stop
type	String	Enumeration	Activity <code>type</code> enumeration	Type of the stop
location.lat	Number	Decimal degrees	Copied from the activity	Latitude of the stop
location.lng	Number	Decimal degrees	Copied from the activity	Longitude of the stop
startTime	String	5 characters, HH:mm	24-hour clock; first stop at 09:00	Scheduled start of the visit
endTime	String	5 characters, HH:mm	<code>startTime + durationHours</code>	Scheduled end of the visit
durationHours	Number	0.5–8	Copied from the activity	Visit time in hours
entryFee	Number	Integer ≥ 0 (INR)	Copied from the activity	Entry fee per person

Field	Data Type	Size / Range	Constraints	Description
travelKmFromPrev	Number	≥ 0	Leg distance returned by the RouteProvider; 0 for the first stop of a day	Distance from the previous stop in km
travelMinutesFromPrev	Number	≥ 0	Leg duration returned by the RouteProvider; 0 for the first stop of a day	Travel time from the previous stop in minutes

3.5.7 Embedded structure: budget

Table 3.14: Data Dictionary — budget (embedded in trips.budget)

Field	Data Type	Size / Range	Constraints	Description
tier	String	Enumeration	Enum: economy, standard, luxury	Tier used for the estimate
currency	String	3 characters	Constant INR	Currency of all amounts
rooms	Number	1–6	Derived: $\text{ceil}(\text{travelers} / 2)$	Number of rooms
nights	Number	1–13	Derived: $\text{max}(\text{days} - 1, 1)$	Number of nights
breakdown.stay	Number	Integer ≥ 0 (INR)	Derived	Accommodation cost of the group
breakdown.food	Number	Integer ≥ 0 (INR)	Derived	Food cost of the group
breakdown.transport	Number	Integer ≥ 0 (INR)	Derived	Local transport cost of the group
breakdown.activities	Number	Integer ≥ 0 (INR)	Derived	Entry fees of the group
total	Number	Integer ≥ 0 (INR)	Derived: sum of the four components	Estimated cost of the trip
perPerson	Number	Integer ≥ 0 (INR)	Derived: $\text{round}(\text{total} / \text{travelers})$	Share of one traveller

3.5.8 Relationships and indexes

Table 3.15: Relationships and Indexes

Collection	Key / index	Purpose
users	Unique index on email	One account per e-mail address; duplicate registration is reported as 409.
destinations	Unique index on slug	Stable, human-readable URL of a destination.
activities	Index on destination; unique compound index on { destination, name }	Fast loading of the activities of a destination; makes the seed script idempotent.

Collection	Key / index	Purpose
trips	Index on user; FK user → users (many trips to one user); FK destination → destinations (many trips to one destination)	Listing the trips of the logged-in user; population of destination data in trip responses.
trips.itinerary.items	FK activity → activities (reference only)	Traceability of a snapshot to its source activity.

3.5.9 Budget computation

The budget is computed by the pure function `estimateBudget` of `budgetService`. It receives the daily cost table of the destination, the tier, the number of days, the number of travellers and the activity fees per person, and has no side effects, which makes it easy to unit-test. The activity fees per person are the sum of the entry fees of all scheduled itinerary items; for the quick estimate (FR-12) they are 0.

```

nights    = max(days - 1, 1)
rooms     = ceil(travelers / 2)
stay      = dailyCost[tier].stay      x rooms      x nights
food      = dailyCost[tier].food      x travelers x days
transport = dailyCost[tier].transport x travelers x days
activities = activityFeesPerPerson    x travelers
total     = stay + food + transport + activities
perPerson = round(total / travelers)

```

Worked example. The demonstration plan is a trip to Manali of 4 days for 2 travellers in the standard tier. The seeded standard-tier daily cost of Manali is Rs. 3,000 per room per night for stay, Rs. 900 per person per day for food and Rs. 900 per person per day for transport. For the illustration it is assumed that the entry fees of the scheduled activities add up to Rs. 1,300 per person; the actual figure depends on the generated itinerary. With three travellers instead of two, the number of rooms would become $\text{ceil}(3 / 2) = 2$, which doubles the stay component.

Table 3.16: Worked Budget Example (Manali, 4 days, 2 travellers, standard tier)

Quantity	Computation	Result
nights	$\text{max}(4 - 1, 1)$	3
rooms	$\text{ceil}(2 / 2)$	1
stay	$3,000 \times 1 \text{ room} \times 3 \text{ nights}$	Rs. 9,000
food	$900 \times 2 \text{ travellers} \times 4 \text{ days}$	Rs. 7,200
transport	$900 \times 2 \text{ travellers} \times 4 \text{ days}$	Rs. 7,200
activities	$1,300 \text{ (assumed)} \times 2 \text{ travellers}$	Rs. 2,600
total	$9,000 + 7,200 + 7,200 + 2,600$	Rs. 26,000
perPerson	$\text{round}(26,000 / 2)$	Rs. 13,000